SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1214	**Course Title:**	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	**Assessment:**	Industry Problem solving task
Weightage:	30%	**Total Marks:**	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	**M.KOUSHIK REDDY**	**Reg. No.:**	**192372185**
Section / Batch:	**CSE-AI**	**Date:**	**17/08/2026**

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A high-frequency financial trading system must process large volumes of market data with extremely low response time. Analyze the roles of CPU caches, DRAM, and persistent storage in handling frequently accessed trading data. Considering latency, data locality, capacity, and bandwidth, propose a memory hierarchy that minimizes data-access delays and supports high transaction throughput.
2. A medical imaging workstation processes large CT and MRI datasets while simultaneously running image enhancement and visualization applications. Evaluate the interaction between cache memory, DRAM, and high-capacity storage during image processing. Propose an optimized memory hierarchy that balances access latency, memory capacity, bandwidth, and cost to improve overall processing performance.
3. A smart surveillance system receives continuous video streams from multiple high-resolution cameras and must detect objects in real time. Analyze the memory requirements for storing video frames, intermediate processing data, and frequently accessed detection parameters. Compare cache, DRAM, and non-volatile memory technologies and design a hierarchy that minimizes memory bottlenecks while maintaining high data throughput.
4. A scientific computing system performs large-scale simulations involving matrices and continuously accessed numerical datasets. Analyze how cache capacity, data locality, cache misses, DRAM bandwidth, and memory latency affect the execution of computational kernels. Recommend memory-hierarchy improvements that can reduce cache misses, increase effective memory bandwidth, and improve overall simulation performance.

5. A wearable health-monitoring device continuously collects sensor readings while operating under strict battery constraints. Analyze the trade-offs among SRAM, DRAM, and non-volatile memory in terms of latency, capacity, energy consumption, and data retention. Design a memory hierarchy that provides rapid access to frequently processed sensor data while minimizing power consumption and extending battery life.

Code of Conduct

*I **M.Koushik Reddy/192372185** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

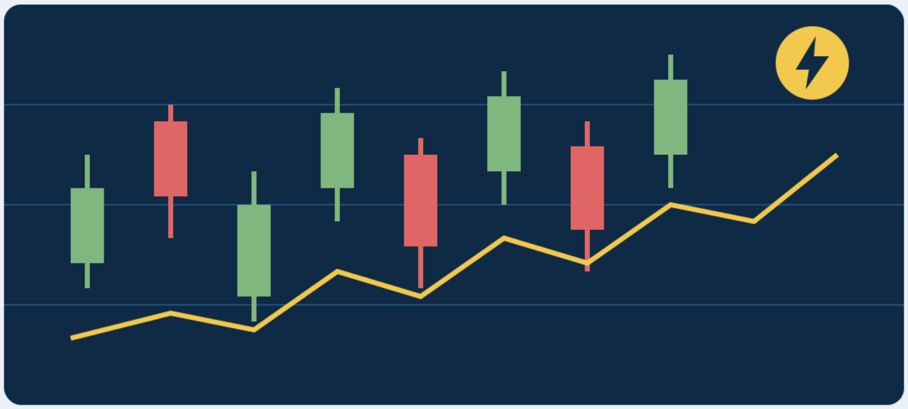
Signature of the Student: ______________________

Course Coordinator Faculty Incharge:__________________________

Case Study 1: High-Frequency Financial Trading System

Question

A high-frequency financial trading system must process large volumes of market data with extremely low response time. Analyze the roles of CPU caches, DRAM, and persistent storage in handling frequently accessed trading data. Considering latency, data locality, capacity, and bandwidth, propose a memory hierarchy that minimizes data-access delays and supports high transaction throughput.



Real-Time Market Data Feed

Analysis

A high-frequency trading system requires very low latency and high throughput because market prices and orders change continuously.

Memory	Role	Latency	Capacity	Usage
CPU Cache (L1/L2/L3)	Stores frequently used market data and instructions	Very low	Small	Active trading calculations
DRAM	Stores order books, recent market data and working datasets	Low	Medium–High	Main working memory
Persistent Storage (NVMe SSD)	Stores historical data, logs and backups	Higher	Very high	Long-term storage

Proposed Memory Hierarchy

L1 Cache
Most frequently accessed data

↓

L2 Cache
Secondary fast cache

↓

L3 Cache
Shared last-level cache

↓

DRAM
Order books / active market data

↓

NVMe SSD
Historical data / transaction logs

Recommendation

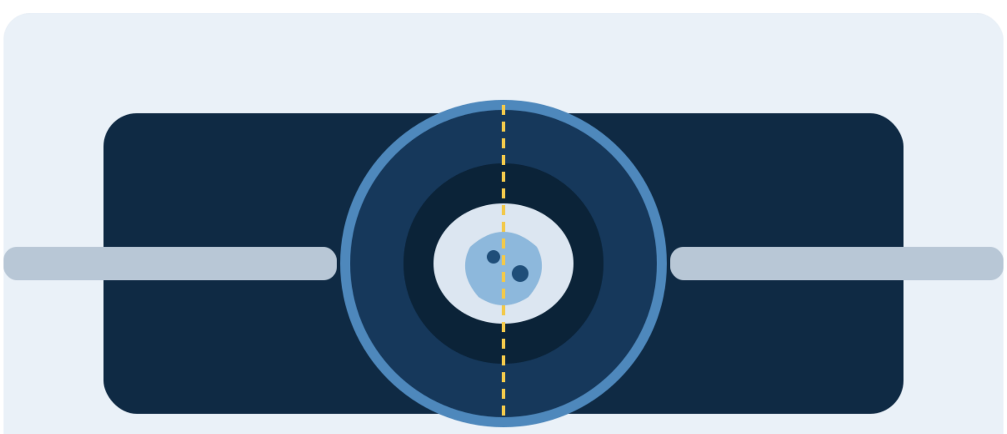
- Keep frequently accessed order-book data in CPU cache.
- Use high-speed DRAM for active market datasets.
- Use NVMe SSD for historical data and transaction logs.
- Apply data locality and cache-friendly data structures.
- Use high-bandwidth memory channels to reduce DRAM bottlenecks.

Result:
This hierarchy minimizes memory-access latency and enables high transaction throughput and rapid trading decisions.

Case Study 2: Medical Imaging Workstation

Question

A medical imaging workstation processes large CT and MRI datasets while simultaneously running image enhancement and visualization applications. Evaluate the interaction between cache memory, DRAM, and high-capacity storage during image processing. Propose an optimized memory hierarchy that balances access latency, memory capacity, bandwidth, and cost to improve overall processing performance.



CT / MRI Imaging & Visualization

Analysis

CT and MRI datasets are very large. Processing them requires fast access to image slices, intermediate results and visualization data.

Memory	Role	Capacity	Speed	Usage
CPU Cache	Frequently processed pixels/data	Very small	Extremely high	Image enhancement calculations
DRAM	Active CT/MRI datasets	High	High	Processing and visualization
NVMe SSD	Complete datasets and applications	Very high	Medium–High	Dataset storage

Proposed Memory Hierarchy

CPU / GPU
Processing units

↓

CPU Cache
Frequently processed image blocks

↓

Large DRAM
Active CT/MRI volumes

↓

High-Speed NVMe
Complete datasets & applications

↓

Large Medical Dataset Archive
Long-term storage

Optimization

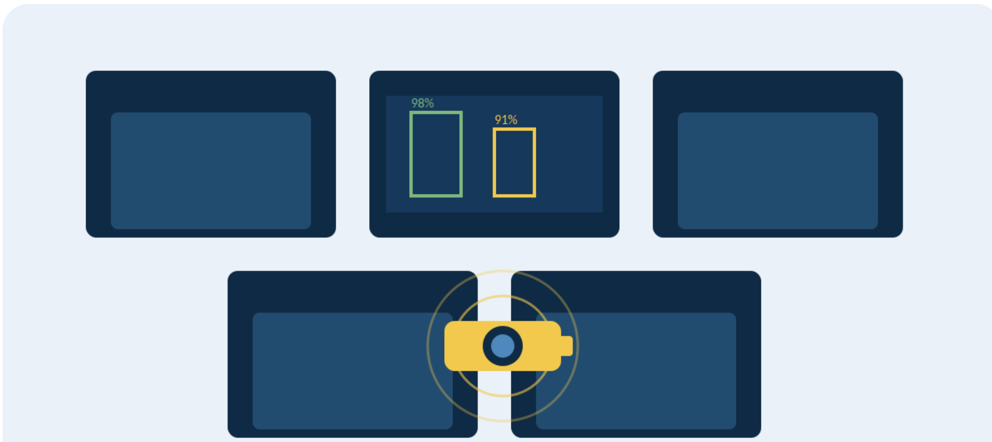
1. Keep frequently processed image blocks in L1/L2/L3 cache.
2. Use sufficient DRAM capacity to hold active CT/MRI volumes.
3. Use high-bandwidth DRAM for image-processing operations.
4. Use NVMe SSD for large datasets and temporary files.
5. Transfer only required image sections into DRAM instead of loading everything repeatedly.

Result:
The optimized hierarchy provides a balance between latency, capacity, bandwidth and cost, improving image enhancement and visualization performance.

Case Study 3: Smart Surveillance System

Question

A smart surveillance system receives continuous video streams from multiple high-resolution cameras and must detect objects in real time. Analyze the memory requirements for storing video frames, intermediate processing data, and frequently accessed detection parameters. Compare cache, DRAM, and non-volatile memory technologies and design a hierarchy that minimizes memory bottlenecks while maintaining high data throughput.



Multi-Camera Real-Time Object Detection

Analysis

Multiple high-resolution cameras continuously generate large amounts of video data. The system must process frames in real time.

Memory	Role	Characteristics	Usage
CPU/GPU Cache	Detection parameters and active processing data	Very fast, small	Object detection
DRAM	Current video frames and intermediate results	Fast, large	Real-time processing
Non-Volatile Memory	Recorded video and configuration	Large, persistent	Long-term storage

Proposed Hierarchy

Cameras
Video stream sources

↓

CPU/GPU Cache
Detection parameters

↓

DRAM
Current frames & intermediate results

↓

NVMe / NVM
Recorded video

Design

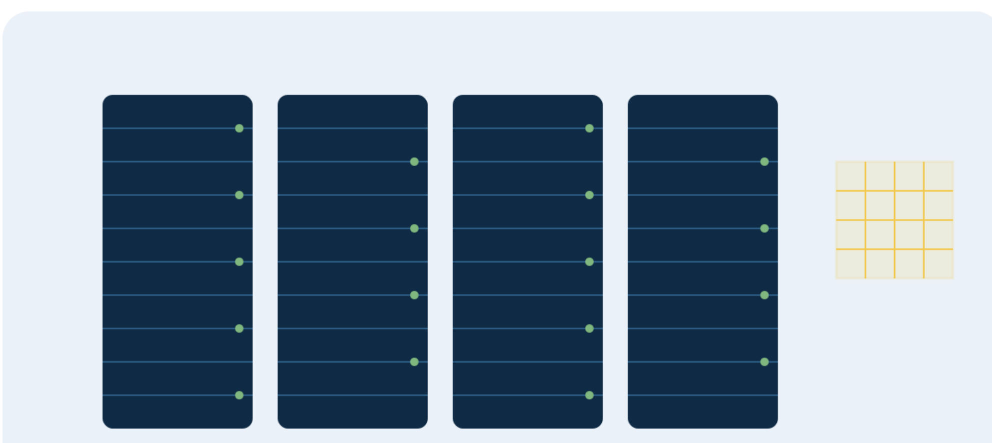
- Store frequently used object-detection parameters in cache.
- Keep current frames and intermediate processing results in DRAM.
- Use high-bandwidth DRAM to support multiple camera streams.
- Store recorded video in NVMe SSD or other non-volatile storage.
- Use buffering to prevent camera data from waiting for storage operations.
- GPU cache/shared memory can be used for parallel video processing.

> **Result:**
> This hierarchy reduces memory bottlenecks and provides the high bandwidth required for real-time object detection.

Case Study 4: Scientific Computing System

Question

A scientific computing system performs large-scale simulations involving matrices and continuously accessed numerical datasets. Analyze how cache capacity, data locality, cache misses, DRAM bandwidth, and memory latency affect the execution of computational kernels.



Large-Scale Matrix Simulation Cluster

Analysis

Scientific simulations perform repeated operations on large matrices and numerical datasets. Data locality and cache efficiency are very important.

Factor	Effect
Cache Capacity	Larger cache can hold more frequently reused data
Data Locality	Improves cache hits
Cache Misses	Increase access time
DRAM Bandwidth	Determines how quickly large datasets can be transferred
Memory Latency	Affects performance when data is not present in cache

Proposed Hierarchy

CPU Cores
Processing units

↓

L1 / L2 Cache
Per-core fast cache

↓

L3 Cache
Shared cache

↓

High-Speed DRAM
Large numerical datasets

↓

Persistent Storage
Simulation data

Optimization

1. Increase effective cache capacity.
2. Use cache blocking/tiling for matrix operations.
3. Organize data for sequential access.
4. Improve spatial and temporal locality.
5. Use high-bandwidth multi-channel DRAM.
6. Reduce unnecessary data movement between cache and DRAM.
7. Use vectorization and parallel processing.

Result:
Better locality and reduced cache misses increase effective memory bandwidth and significantly improve simulation execution time.

Case Study 5: Wearable Health-Monitoring Device

Question

A wearable health-monitoring device continuously collects sensor readings while operating under strict battery constraints. Analyze the trade-offs among SRAM, DRAM, and non-volatile memory in terms of latency, capacity, energy consumption, and data retention. Design a memory hierarchy that provides rapid access to frequently processed sensor data while minimizing power consumption and extending battery life.

Continuous Sensor Monitoring on a Battery Budget

Analysis

Wearable devices continuously collect sensor data but have limited battery capacity. Memory selection must therefore consider energy consumption as well as speed and capacity.

Memory	Latency	Capacity	Power	Data Retention
SRAM	Very low	Small	Relatively high when active	Volatile
DRAM	Low	Medium/Large	Requires refresh	Volatile
Non-Volatile Memory	Higher	Large	Low standby	Retains data

Proposed Memory Hierarchy

Sensors
Data sources

↓

SRAM
Current sensor readings

↓

DRAM
Temporary data / buffer

↓

Flash / NVM
Long-term data

Design

- Store current sensor readings and active calculations in SRAM.
- Use a small DRAM buffer only when larger temporary datasets are required.
- Store historical sensor readings in Flash/NVM.
- Put unused memory components into low-power/sleep modes.
- Process data locally to reduce unnecessary memory transfers.
- Store only important or compressed sensor data to reduce storage and energy usage.

Result:
The proposed hierarchy provides fast access to active sensor data while reducing power consumption, thereby extending the wearable device's battery life.